

DESARROLLO DE INTERFACES · 0488

Corporate App Ionic + Angular Standalone

Productos con vendedor · Publicación en GitHub y Vercel

Alumno	Jorge González Gómez
Curso	2º DAM — Universidad Nebrija
Sesión	02-10-2026
Entrega	05-10-2026
Repositorio	https://github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2
Vercel	https://desarrollo-interfaces-dam-2.vercel.app

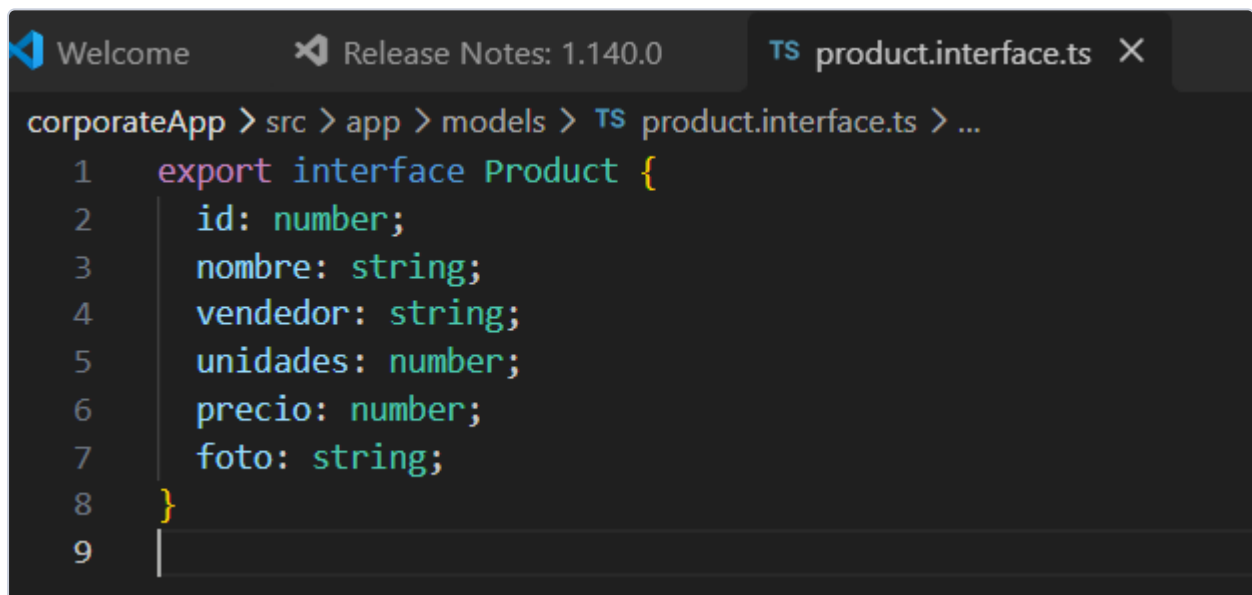
Objetivo

Aplicación corporativa creada con **ionic** y **Angular standalone** (`ionic start corporateApp blank --type=angular --standalone`), con una página Home con menú horizontal y una página Productos que lee los datos de un JSON local a través de un servicio (`ProductsService`) inyectado en la página mediante **inyección de dependencias**.

En esta entrega se pide: añadir a los productos un **vendedor** (texto), mostrar **al menos tres productos** con ese dato en el grid, y **publicar** el proyecto en GitHub y Vercel.

1. Modelo de producto con el campo vendedor

El enunciado pide que cada producto tenga un **vendedor de tipo string**. El primer cambio es en el **modelo** (`src/app/models/product.interface.ts`): la interfaz `Product` define la forma de un producto y ahora incluye `vendedor: string;`. A partir de aquí TypeScript exige que todo producto tenga vendedor y avisa si falta.



The screenshot shows a code editor with a dark theme. The top bar includes a 'Welcome' message, 'Release Notes: 1.140.0', and a tab for 'TS product.interface.ts'. The breadcrumb navigation shows the path: 'corporateApp > src > app > models > TS product.interface.ts > ...'. The code defines an interface 'Product' with the following properties: 'id: number;', 'nombre: string;', 'vendedor: string;', 'unidades: number;', 'precio: number;', and 'foto: string;'. The interface is enclosed in curly braces and is preceded by 'export interface'.

```
1 export interface Product {
2   id: number;
3   nombre: string;
4   vendedor: string;
5   unidades: number;
6   precio: number;
7   foto: string;
8 }
9
```

Captura 1 — Interfaz Product con el nuevo campo `vendedor: string`.

2. Datos: tres productos con su vendedor

Los productos se leen de un fichero JSON local (`src/assets/data/products.json`), que hace de base de datos. Se han añadido el campo `"vendedor"` a cada producto y un tercer producto (Teclado Logitech, con su foto en `assets/images/logitech.jpg`), cumpliendo el mínimo de tres productos.

```
CorporateApp > src > assets > data > {} products.json > ...
1  [
2    {
3      "id": 1,
4      "nombre": "Portátil Dell",
5      "vendedor": "Ana García",
6      "unidades": 12,
7      "precio": 1200,
8      "foto": "assets/images/dell.jpg"
9    },
10   {
11     "id": 2,
12     "nombre": "Monitor LG",
13     "vendedor": "Luis Martín",
14     "unidades": 8,
15     "precio": 299,
16     "foto": "assets/images/lg.jpg"
17   },
18   {
19     "id": 3,
20     "nombre": "Teclado Logitech",
21     "vendedor": "Carlos Pérez",
22     "unidades": 25,
23     "precio": 89,
24     "foto": "assets/images/logitech.jpg"
25   }
26 ]
27
```

Captura 2 — `products.json` con 3 productos y sus vendedores.

3. Grid actualizado con la columna Vendedor

En la plantilla de la página (`productos.page.html`) se añade una columna más en la cabecera (`Vendedor`) y otra en la fila que se repite con `*ngFor` (`{{ product.vendedor }}` , interpolación). El grid de Ionic se divide en 12 partes, así que se fija el ancho de cada columna con `size` para que las seis quepan también en un móvil ($1 + 2.5 + 2.5 + 1.5 + 2 + 2.5 = 12$).

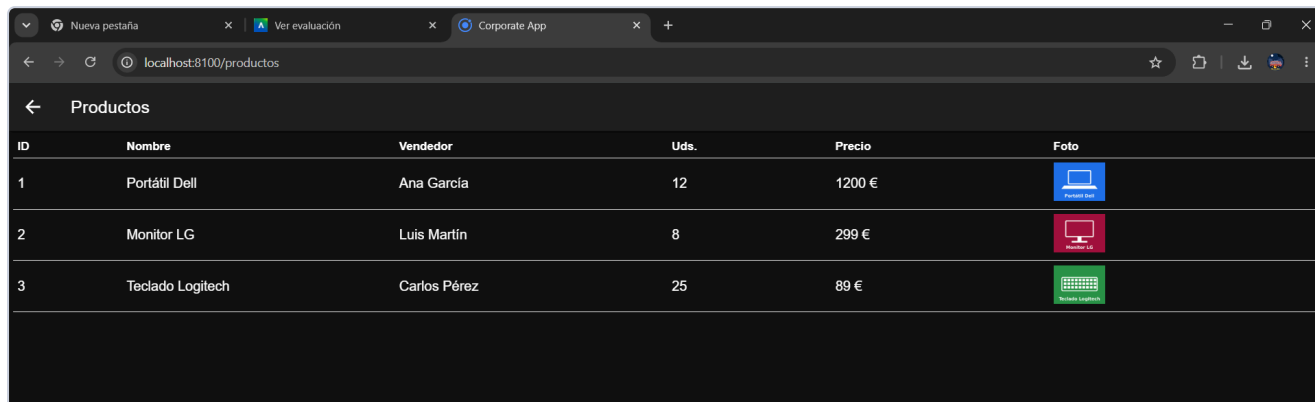
El servicio (`ProductsService`) y la página `.ts` **no cambian**: la página recibe el servicio por **inyección de dependencias** (`constructor(private productService: ProductsService)`) y este devuelve lo que haya en el JSON. Al estar separadas las capas (modelo, datos, servicio y vista), el cambio solo afecta a modelo, datos y vista.

```
corporateApp > src > app > pages > productos > productos.page.html > ion-header
1 <ion-header>
2   <ion-toolbar>
3     <ion-buttons slot="start">
4       <ion-back-button defaultHref="/home"></ion-back-button>
5     </ion-buttons>
6     <ion-title>Productos</ion-title>
7   </ion-toolbar>
8 </ion-header>
9
10 <ion-content>
11
12   <ion-grid>
13
14     <ion-row class="header">
15       <ion-col size="1">ID</ion-col>
16       <ion-col size="2.5">Nombre</ion-col>
17       <ion-col size="2.5">Vendedor</ion-col>
18       <ion-col size="1.5">Uds.</ion-col>
19       <ion-col size="2">Precio</ion-col>
20       <ion-col size="2.5">Foto</ion-col>
21     </ion-row>
22
23     <ion-row *ngFor="let product of products()">
24       <ion-col size="1">{{ product.id }}</ion-col>
25       <ion-col size="2.5">{{ product.nombre }}</ion-col>
26       <ion-col size="2.5">{{ product.vendedor }}</ion-col>
27       <ion-col size="1.5">{{ product.unidades }}</ion-col>
28       <ion-col size="2">{{ product.precio }} €</ion-col>
29       <ion-col size="2.5">
30         <img [src]="product.foto" width="60" />
31       </ion-col>
32     </ion-row>
33
34   </ion-grid>
35
36 </ion-content>
```

Captura 3 — `productos.page.html` con la columna Vendedor en cabecera y filas.

4. Resultado en el navegador

Con `ionic serve` la aplicación se ejecuta en `http://localhost:8100`. En la página **Productos** se muestran los tres productos con su ID, nombre, vendedor, unidades, precio y foto.



ID	Nombre	Vendedor	Uds.	Precio	Foto
1	Portátil Dell	Ana García	12	1200 €	
2	Monitor LG	Luis Martín	8	299 €	
3	Teclado Logitech	Carlos Pérez	25	89 €	

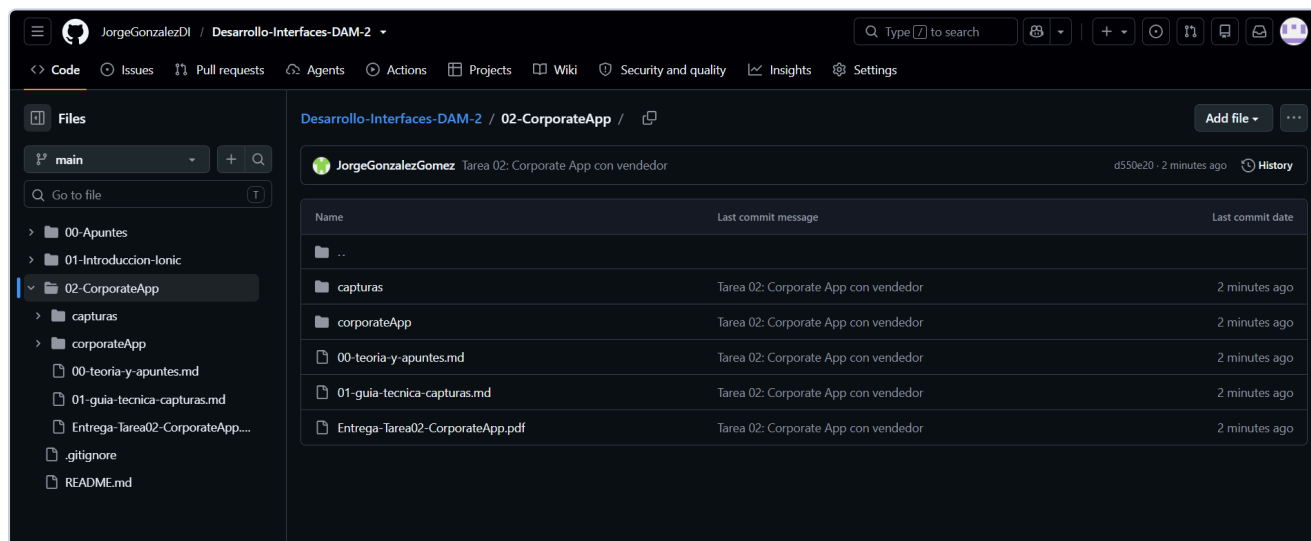
Captura 4 — Página Productos en `localhost:8100/productos` con el vendedor de cada producto.

5. Publicación en GitHub

El proyecto está en el repositorio de la asignatura (github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2), carpeta `02-CorporateApp/corporateApp`. Se suben los cambios con `git add .`, `git commit -m "..."` y `git push`. Las carpetas `node_modules`, `.angular` y `www` no se suben (están en `.gitignore`).

```
PS C:\DAM 2\Desarrollo de Interfaces> git push
Enumerating objects: 79, done.
Counting objects: 100% (79/79), done.
Delta compression using up to 12 threads
Compressing objects: 100% (69/69), done.
Writing objects: 100% (75/75), 523.27 KiB | 7.07 MiB/s, done.
Total 75 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2.git
    222ed7d..d550e20  prueba -> prueba
PS C:\DAM 2\Desarrollo de Interfaces>
```

Captura 5 — `git add`, `commit` y `push` en la terminal.

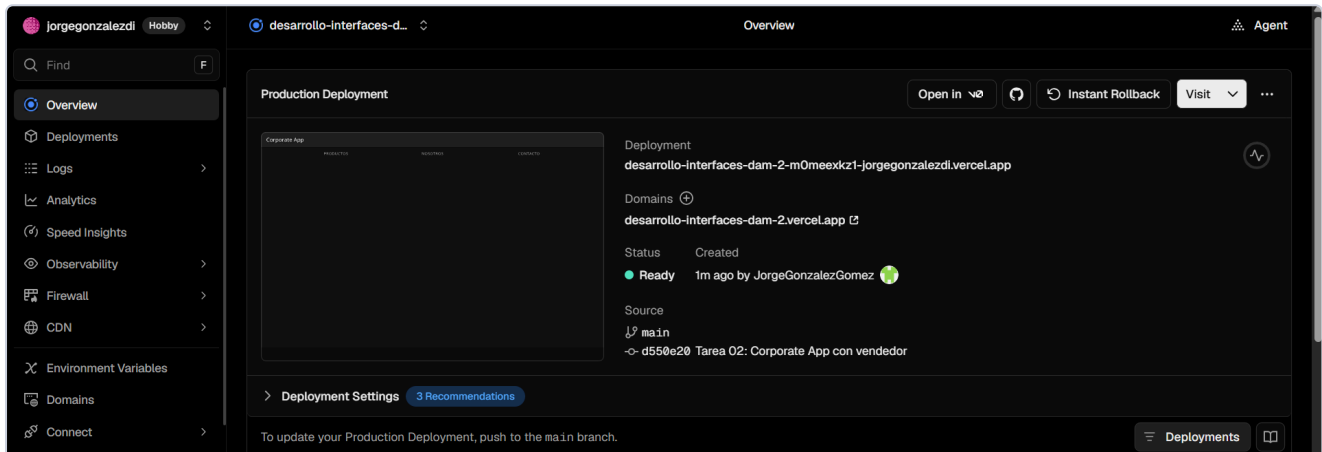


Captura 6 — Repositorio en GitHub (rama main) con la carpeta 02-CorporateApp.

6. Despliegue en Vercel

Vercel importa el repositorio de GitHub, ejecuta `npm run build` y publica la carpeta `www`. Configuración: *Framework Preset* Other, *Root Directory* 02-CorporateApp/corporateApp, *Output Directory* `www`. Se añadió `vercel.json` para que, al ser una SPA, cualquier ruta (p. ej. `/productos`) sirva `index.html`, y `"engines": {"node": "24.x"}` en `package.json` porque Angular 22 necesita Node moderno. Cada `git push` genera un nuevo despliegue automáticamente.

URL pública: <https://desarrollo-interfaces-dam-2.vercel.app>



Captura 7 — Despliegue de producción en Vercel en estado Ready, desde la rama main.